1. Which optimization produced the greatest improvement?

Ans: The heartbeat mechanism with dead-client detection and cleanup produced the greatest improvement. It made the server more reliable by automatically detecting inactive clients and removing them from the active session list. This prevented stale connections from staying in memory and improved the accuracy of the online user list. It also helped the server remain stable during longer test runs.

2. Why is scalability important?

Ans: Scalability is important because a network application should continue working properly when the number of users increases. If a system works only for a few users but becomes slow or unstable with more clients, it is not practical. In this assignment, scalability was tested with 5, 8, and 10 concurrent clients. This showed whether the server could handle growth without major performance loss.

3. What reliability issues did you discover?

Ans: The main reliability issues were stale client connections, unexpected disconnections, and the need

for proper cleanup after a session ends. Without heartbeat monitoring, the server could keep inactive clients in memory and still think they were online. That would make the online user list inaccurate and waste server resources. These issues showed that the system needed better failure handling and recovery.

4. Which optimization was the most difficult?
Ans: The most difficult optimization was the combination of heartbeat handling, stale-client reaping, and automatic reconnection. These features were difficult because they involved timing, background activity, and connection state at the same time. A small mistake could incorrectly disconnect active clients or fail to remove dead ones. This part required the most attention during testing and debugging.

5. What additional improvements are required to support 100 users?
Ans: To support 100 users, the application would need stronger scalability improvements than the current design. The thread-per-client model may become heavy when the number of users grows much larger.
A better design would use asynchronous I/O or a

worker pool to handle connections more efficiently. The system would also need better resource management, stronger monitoring, and possibly load balancing.